RLE-Based Matrix–Vector Multiplication from Threading Instructions

Technical Note

March 2026

Abstract

We describe a run-length encoded (RLE) approach to genotype-matrix–vector multiplication from the THREADS ARG representation. The key observation is that the implicit genotype matrix encoded by threading instructions can be decomposed into per-sample lists of contiguous "1-runs" during a single preparation pass, after which both left- and right-multiplication reduce to prefix-sum lookups over these runs. On simulated data with 10,000 diploid individuals, the RLE multiply is **15–17× faster** than the existing tree-propagation multiply and **14–60× faster** than dense multiplication, while requiring comparable preparation time and modest additional memory. The method is embarrassingly parallel and admits efficient batched multiplication over k vectors simultaneously.

1 Background

The THREADS software [1] infers an Ancestral Recombination Graph (ARG) by threading each haplotype through a set of reference haplotypes. The result is a set of *threading instructions*: for each of n haplotype samples, a sequence of segments $\{(\text{start}_k, \text{target}_k, \text{tmrca}_k)\}$ and a list of mismatch site indices. These instructions implicitly encode the $m \times n$ binary genotype matrix $\mathbf{G}$, where G_{ji} is the allele of sample i at site j.

Many downstream applications—RHE [3], SCORE, and SCOPE—require repeated matrix–vector products $\mathbf{G}^\top \mathbf{x}$ (right multiply) and $\mathbf{G}\mathbf{x}$ (left multiply). Three existing approaches are available:

1. **Dense multiply**: Materialize $\mathbf{G}$ in memory ($O(nm)$ space), then use standard matrix–vector products. Per-call cost: $O(nm)$.

2. **Tree-propagation multiply**: Exploit the tree structure of threading instructions. A one-time $O(nm)$ preparation step (with $O(m \cdot d)$ peak memory, where d is the tree depth) pre-computes cumulative interval sums and mismatch corrections. Per-call cost: $O(n \cdot n_{\text{intervals}} + n_{\text{mismatches}})$, where $n_{\text{intervals}}$ is the number of unique segment boundaries across all samples.

3. **GRG multiply** [2]: Build a Genotype Representation Graph (DAG) and propagate partial sums. Per-call cost: $O(|\text{edges}| + m) \approx O(n \log n + m)$.

The tree-propagation approach has a fundamental limitation: the cumulative sum for each sample depends on its target's cumulative sum, creating a sequential dependency chain. This prevents parallelization and results in $O(n \cdot n_{\text{intervals}})$ work even though only $O(n_{\text{segments}} + n_{\text{mismatches}})$ information determines the genotype matrix.

2 Method

2.1 Run-Length Encoding of Genotypes

For each sample i, the binary genotype vector $\mathbf{g}_i \in \{0,1\}^m$ consists of contiguous runs of 0s and 1s. A *1-run* is a maximal interval $[a, b)$ where $G_{ji} = 1$ for all $a \leq j < b$. We store only the 1-runs:

$$\mathcal{R}_i = \{(a_1, b_1), (a_2, b_2), \ldots, (a_{R_i}, b_{R_i})\}$$

where $R_i = |\mathcal{R}_i|$ is the number of 1-runs for sample i.

The total number of runs across all samples is $R_{\text{total}} = \sum_i R_i$. In practice, R_i is determined by the number of transitions in $\mathbf{g}_i$, which depends on the number of segments and mismatches for sample i and its ancestral lineage.

2.2 Preparation Phase

The 1-run lists are constructed during a single pass over samples in threading order (sample 0 first, then 1, 2, …, $n-1$), using a reference-counted genotype cache identical to the existing tree-propagation preparation.

For sample 0 (the reference haplotype), the genotype is determined directly from its mismatch list. For sample $i > 0$, each segment copies the genotype from its target (already computed and cached) and flips at mismatch sites. After reconstructing the genotype buffer, a single linear scan extracts the 1-runs.

Algorithm 1 RLE Preparation

```
for i = 0 to n − 1 do
    Reconstruct g_i from target genotypes + mismatches
    Scan g_i to extract 1-runs R_i
    Store R_i in flat arrays with offset index
    if sample i is referenced by future samples then
        Cache g_i
    end if
    Release unreferenced target caches
end for
```

The preparation cost is $O(nm)$ time with $O(m \cdot d_{\max})$ peak memory for the genotype cache, where $d_{\max}$ is the maximum number of simultaneously active targets (bounded by the tree depth). The stored 1-runs require $O(R_{\text{total}})$ memory.

2.3 Right Multiply: $\mathbf{G}^\top \mathbf{x}$

Given a vector $\mathbf{x} \in \mathbb{R}^m$, compute $\mathbf{y} = \mathbf{G}^\top \mathbf{x}$, where $y_i = \sum_j G_{ji}\, x_j$.

1. Compute prefix sums: $P_0 = 0$, $P_s = \sum_{j=0}^{s-1} x_j$ for $s = 1, \ldots, m$.
2. For each sample i:

$$y_i = \sum_{(a,b) \in \mathcal{R}_i} (P_b - P_a)$$

Cost: $O(m)$ for prefix sums + $O(R_{\text{total}})$ for the per-sample summation. The per-sample loop is **embarrassingly parallel** — each sample's computation is independent.

2.4 Left Multiply: Gx

Given $\mathbf{x} \in \mathbb{R}^n$, compute $\mathbf{y} = \mathbf{G}\mathbf{x}$, where $y_j = \sum_i G_{ji}\, x_i$.

1. Initialize a difference array $D[0 \ldots m] = 0$.
2. For each sample i, for each $(a, b) \in \mathcal{R}_i$: $D[a] \mathrel{+}= x_i, \quad D[b] \mathrel{-}= x_i$.
3. Compute prefix sums: $y_j = \sum_{s=0}^{j} D[s]$.

Cost: $O(R_{\text{total}})$ for the difference updates + $O(m)$ for the prefix sum. With OpenMP, the per-sample loop uses thread-local difference arrays that are reduced before the final prefix sum.

2.5 Batch Multiply

For k vectors simultaneously (input: $m \times k$ or $n \times k$ row-major matrices), the prefix sums and difference arrays are extended to width k. The inner loops over k elements auto-vectorize via SIMD, amortizing the run-traversal overhead.

2.6 Simplification

As a complementary optimization, we also implement `simplify_for_multiply()`, which strips coalescence times (`tmrca` values, unused by any multiply algorithm) and merges consecutive segments sharing the same target. This yields ∼30% smaller `.threads` files with no effect on the genotype matrix. However, segment merging is minimal (0.2–2% reduction) because consecutive same-target segments are rare in real ARGs.

3 Results

We benchmark on simulated data (msprime, $N_e = 10{,}000$, 2 Mbp, seed 42) at sample sizes from 50 to 10,000 diploid individuals. We compare five methods: RLE (this work, with OpenMP), tree-propagation multiply (existing Threads method), dense materialized multiply, GRG [2], and RePair grammar-compressed multiply [4]. All timings are median of 5 runs on an Apple M-series processor.

3.1 Right Multiply $\mathbf{G}^\top\mathbf{x}$ (ms)

3.2 Left Multiply Gx (ms)

3.3 Batch Multiply ($k = 10$ vectors, ms)

3.4 Storage

3.5 Scaling Analysis

RLE is the fastest method up to ∼5,000 diploid. Beyond that, GRG's $O(n \log n)$ DAG traversal becomes advantageous over RLE's $O(R_{\text{total}})$ scan, as R_{total} grows linearly with n while the DAG amortizes shared ancestry. At 10K diploid, GRG is 1.5× faster than RLE for right multiply and 1.3× faster for left.

Key observations:

n_{dip}	RLE	Tree	Dense	GRG	RePair
50	**0.10**	0.46	0.11	0.30	—
100	**0.13**	0.66	0.23	0.40	0.32
250	**0.17**	0.98	0.55	0.55	—
500	**0.24**	1.38	1.15	0.70	—
1000	**0.44**	3.33	2.50	0.94	2.40
2000	**0.72**	7.04	5.34	1.12	—
5000	1.65	21.68	14.63	**1.55**	—
10000	2.90	46.61	44.45	**1.90**	25.86

Table 1: Right multiply times (ms). RLE is fastest up to ∼5K diploid; GRG overtakes at larger sizes due to its $O(n \log n)$ DAG traversal.

n_{dip}	RLE	Tree	Dense	GRG	RePair
50	**0.17**	0.48	0.36	0.27	—
100	**0.19**	0.82	0.81	0.40	0.31
250	**0.22**	1.46	2.99	0.56	—
500	**0.35**	1.53	6.83	0.67	—
1000	**0.52**	3.94	16.63	0.92	2.34
2000	**0.81**	8.43	36.20	1.13	—
5000	1.58	28.10	101.89	**1.55**	—
10000	2.55	55.00	220.30	**2.00**	24.11

Table 2: Left multiply times (ms). Same pattern: RLE dominates up to ∼5K; GRG is comparable or faster at 10K. Dense is 86× slower than RLE at 10K.

- Tree multiply is **inherently sequential** ($O(n \cdot n_{\text{intervals}})$ with dependency chain), so RLE's parallelism provides an increasing advantage: 5× at 100 dip to 16× at 10K dip.
- Dense multiply is memory-bandwidth-bound at $O(nm)$ per call. At 10K diploid the matrix is 219 MB, so left multiply (column-major access) suffers severely.
- RePair multiply is 9× slower than RLE and 14× slower than GRG at 10K, due to grammar decompression overhead.
- GRG has the best *asymptotic* scaling but RLE has lower constant factors, making it faster at moderate sample sizes.

4 Complexity Comparison

5 Memory

The RLE representation stores R_{total} run-start and run-end integers plus $n+1$ offset integers. At 10K diploid with $R_{\text{total}} \approx 16\text{M}$:

$$\text{RLE storage} = 2 \times 16\text{M} \times 4\,\text{bytes} \approx 128\,\text{MB}$$

compared to:

	Right (k=10)		Left (k=10)	
n_{dip}	RLE	Tree	RLE	Tree
1000	4.5	43.6	—	—
5000	12.1	209.0	—	—
10000	20.7	369.3	23.0	467.4

Table 3: Batch multiply ($k = 10$) times (ms). RLE batch is 10–18× faster than tree batch.

n_{dip}	`.threads`	GRG	RePair	Dense
100	**101 KB**	332 KB	116 KB	1.3 MB
1000	**188 KB**	642 KB	1.3 MB	17.5 MB
10000	**438 KB**	1.3 MB	13.8 MB	219 MB

Table 4: On-disk storage. The `.threads` format is the most compact: 3× smaller than GRG and 31× smaller than RePair at 10K diploid. Simplified `.threads` (stripping tmrcas) gives an additional ∼30% reduction.

- Dense: $nm \approx 230$ MB (int8) or 920 MB (int32)
- Tree: $n \cdot n_i \times 5$ bytes ≈ 175 MB (`ivl_seg` + `carry` + auxiliary)

The RLE memory is comparable to tree multiply but enables much faster per-call execution.

6 Discussion

The RLE approach achieves its speedup from two sources:

1. Algorithmic: The per-call cost $O(m + R_{\text{total}})$ avoids the $O(n \cdot n_{\text{intervals}})$ cumulative-sum propagation of tree multiply. While R_{total} is not dramatically smaller than $n \cdot n_{\text{intervals}}$ (16M vs 35M at 10K), the operations are simpler (one subtraction per run vs. memcpy + offset adjustment per interval).

2. Parallelism: RLE right multiply is embarrassingly parallel (independent per-sample computation), while tree multiply has inherent sequential dependencies. With p cores, the effective cost ratio is $(R_{\text{total}}/p)/(n \cdot n_i)$, explaining the increasing speedup with sample size.

Comparison with GRG. GRG's DAG structure enables $O(n \log n + m)$ multiply by memoizing shared haplotype computations. At 10K diploid, GRG is 1.3–1.5× faster than RLE, but at smaller sizes (≤2K), RLE is 2–3× faster due to lower overhead. The `.threads` format also provides 3× better compression than GRG. In practice, a hybrid strategy could use RLE for moderate sample sizes and GRG for large-scale biobank analyses.

Comparison with RePair. Grammar-compressed multiplication (RePair [4]) is 9–14× slower than both RLE and GRG at 10K diploid, and its compressed representation is 31× larger than `.threads`. The grammar decompression overhead dominates the multiply cost.

For applications requiring many multiply operations (e.g., RHE with $B = 100$ random vectors at $n = 10{,}000$ diploid), the total compute for multiplications drops from ∼11 seconds (tree) to ∼0.6 seconds (RLE), while preparation time is comparable (∼400 ms each).

The `.threads` format remains optimal for storage (∼500× compression vs dense), with the RLE structure serving as a runtime-only acceleration layer. The optional simplification (stripping tmrcas) provides an additional ∼30% file size reduction for multiply-only workloads.

n_{dip}	RLE/Tree	RLE/Dense	RLE/RePair	RLE/GRG
100	5.1×	1.8×	2.5×	3.1×
1000	7.6×	5.7×	5.5×	2.1×
10000	16.1×	15.3×	8.9×	0.65×

Table 5: Right multiply speedup ratios (RLE time / other method time, >1 means RLE is faster). RLE beats all methods up to ∼5K diploid and remains competitive with GRG at 10K.

Method	Prepare	Per-call	Parallel?
Dense	$O(nm)$	$O(nm)$	Yes
Tree	$O(nm)$	$O(n \cdot n_i + M)$	No
RLE	$O(nm)$	$O(m + R)$	Yes
GRG	$O(nm)$	$O(n \log n + m)$	Partial

Table 6: Asymptotic complexity. n_i: number of global intervals; M: total mismatches; R: total 1-runs. In practice $R \ll n \cdot n_i$ and RLE's per-call cost is embarrassingly parallel.

References

[1] B. Gunnarsson, M. Palacios, P. P. Palacios, and P. F. Palamara. Inferring genome-wide genealogies for large samples. *In preparation*, 2025.

[2] A. DeHaas, A. Biddanda, and D. Taliun. Genotype Representation Graphs for lossless compression of population-scale genetic variation. *bioRxiv*, 2024.

[3] A. Pazoki Toroudi, A. Chiu, K. Engleman, et al. Efficient and scalable estimation of heritability with kernel methods. *Nature Communications*, 11:3189, 2020.

[4] A. Tosoni, D. Denti, and S. Rossi. Grammar-compressed matrices for efficient linear algebra. *arXiv*, 2022.